BudgetFlow — Personal Finance Tracker

A full-stack personal budget tracker built with React + Flask + SQLite.

Features

- **Auth** — Register/login with JWT-style session tokens
- **Onboarding** — Set monthly salary and create custom budget categories (fixed ₹ or % of salary)
- **Transactions** — Add income/expense entries tied to your categories
- **Budget tracking** — Visual budget bars per category with over-budget warnings
- **Monthly trend** — Bar chart showing income vs expense over last 6 months
- **Search & Filter** — Filter transactions by keyword, category, or type
- **CSV Export** — Download all transactions as a CSV file

Tech Stack

Layer	Technology
Frontend	React 18 + Vite
Backend	Python 3 + Flask
Database	SQLite (via sqlite3)
Auth	Token-based sessions

Project Structure

```
project-bt/
├── backend/
│   ├── app.py          # Flask API
│   ├── budget.db       # SQLite database (auto-created)
│   └── venv/           # Python virtual environment
└── frontend/
    └── src/
        └── App.jsx     # React single-page app
```

Setup & Run

Backend

```bash
cd backend
python -m venv venv
venv\Scripts\activate        # Windows
source venv/bin/activate     # Mac/Linux
pip install flask flask-cors
python app.py                # Runs on port 8000
```

Frontend

```bash
cd frontend
npm install
npm run dev                  # Runs on port 5173
```

Open `http://localhost:5173`

API Endpoints

Method	Endpoint	Description
POST	/api/auth/register	Register new user
POST	/api/auth/login	Login
POST	/api/auth/logout	Logout
POST	/api/onboarding	Save categories & budgets
GET	/api/categories	Get user's categories
GET	/api/transactions	List transactions (supports search/filter)
POST	/api/transactions	Add transaction
DELETE	/api/transactions/:id	Delete transaction

Method	Endpoint	Description
GET	/api/summary	Dashboard summary + budget status
GET	/api/export/csv	Export transactions to CSV

Key Technical Decisions

1. **SQLite over PostgreSQL** — Zero config, file-based, perfect for a local assessment project. Easily swappable to PostgreSQL with minimal changes.
2. **Token auth without JWT library** — Used `secrets.token_hex` + a sessions table instead of full JWT to keep dependencies minimal while still being stateless from the client's perspective.
3. **Budget as fixed or % of salary** — Percentage-type budgets are converted to fixed amounts at onboarding time, making summary queries simple and fast.
4. **Single-file React** — All components in `App.jsx` for simplicity in assessment context. In production this would be split into components/pages/hooks.
5. **In-memory filtering on backend** — Search and filter applied at SQL level for efficiency, not client-side.

Extension Approach

- **Database** → Swap SQLite for PostgreSQL using `psycopg2` or SQLAlchemy ORM
- **Auth** → Replace session tokens with proper JWT (PyJWT)
- **Recurring transactions** → Add `recurrence` field + cron job
- **Multiple budgets** → Monthly budget resets with history tracking
- **Charts** → Integrate Recharts or Chart.js for richer visualizations
- **Mobile** → Convert to React Native or wrap in Capacitor

AI Usage

This project was built with AI assistance (Claude by Anthropic):

- Initial boilerplate and project structure
- CORS debugging and Flask configuration
- Component architecture decisions

- SQL query design for summary/budget endpoints

See claude.md for the full AI guidance file used.